

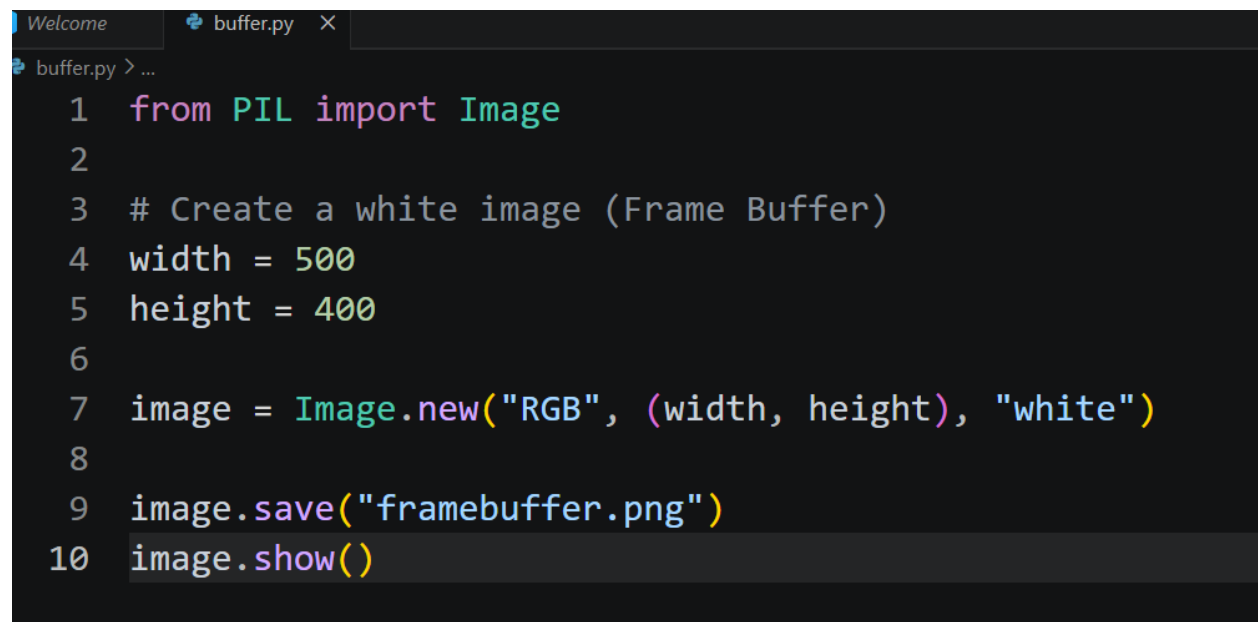
COMPUTER GRAPHICS PRACTICALS

OBJECTIVES

BUFFERS, PIXELS, PLOTTING INDIVIDUAL PIXELS, DRAWING BASIC SHAPES, COLOR MODELS, AND THE USE OF PILLOW FOR DRAWING

Practical 1: Creating a Blank Image (Frame Buffer)

A frame buffer is simply an area in memory where every pixel of an image is stored.



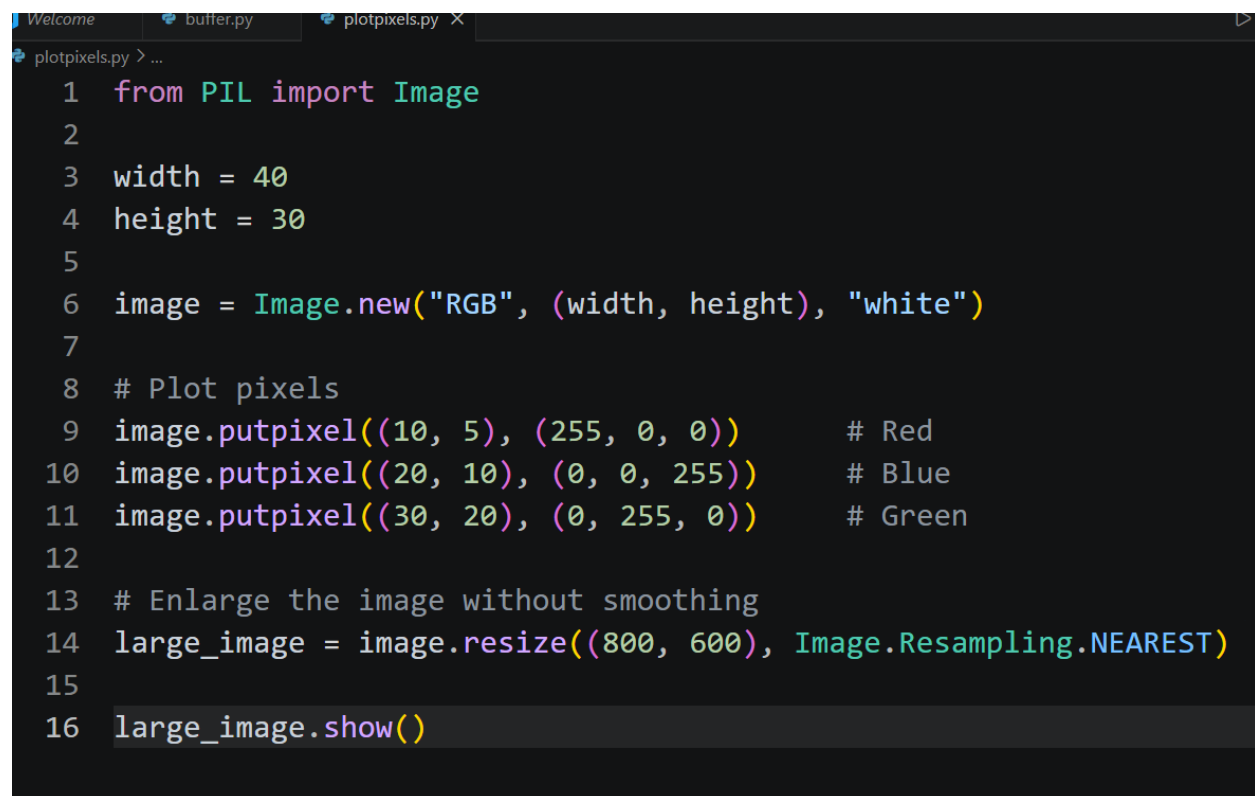
```
buffer.py > ...  
1  from PIL import Image  
2  
3  # Create a white image (Frame Buffer)  
4  width = 500  
5  height = 400  
6  
7  image = Image.new("RGB", (width, height), "white")  
8  
9  image.save("framebuffer.png")  
10 image.show()
```

Explanation

The statement **from PIL import Image** imports the Image class from the **Pillow library**, allowing Python to create, edit, and save images. The variables **width = 500** and **height = 400** define **the size of the image**, making it 500 pixels wide and 400 pixels high. *A pixel is the smallest unit of a digital image, so these values determine the image's dimensions.*

The line **image = Image.new("RGB", (width, height), "white")** creates a new blank image using the **RGB color model** and **fills every pixel** with the **color white**. The image is stored in the variable **image** so it can be used later. Next, **image.save("framebuffer.png")** saves the image to the computer with the filename **framebuffer.png**, while **image.show()** opens the image in the computer's default image viewer so you can immediately see the blank white image that has been created.

Practical 2: Plotting Individual Pixels

A screenshot of a code editor with a dark background. The editor has three tabs at the top: 'Welcome', 'buffer.py', and 'plotpixels.py'. The 'plotpixels.py' tab is active. The code is as follows:

```
1 from PIL import Image
2
3 width = 40
4 height = 30
5
6 image = Image.new("RGB", (width, height), "white")
7
8 # Plot pixels
9 image.putpixel((10, 5), (255, 0, 0))      # Red
10 image.putpixel((20, 10), (0, 0, 255))    # Blue
11 image.putpixel((30, 20), (0, 255, 0))    # Green
12
13 # Enlarge the image without smoothing
14 large_image = image.resize((800, 600), Image.Resampling.NEAREST)
15
16 large_image.show()
```

Explanation

This program first creates a small blank image that is **40 pixels wide** and **30 pixels high** using the Pillow library. It then uses the `putpixel()` function to plot three individual pixels at different positions: a **red pixel** at `(10, 5)`, a **blue pixel** at `(20, 10)`, and a **green pixel** at `(30, 20)`. This demonstrates how individual pixels can be placed at specific coordinates to create simple graphics.

After plotting the pixels, the program enlarges the image to **800 × 600 pixels** using ***Image.Resampling.NEAREST***. This resizing method enlarges each pixel without smoothing or blending, making every pixel appear as a larger square. Finally, ***large_image.show()*** displays the enlarged image, making it easier to see the individual colored pixels and understand how pixel plotting works.

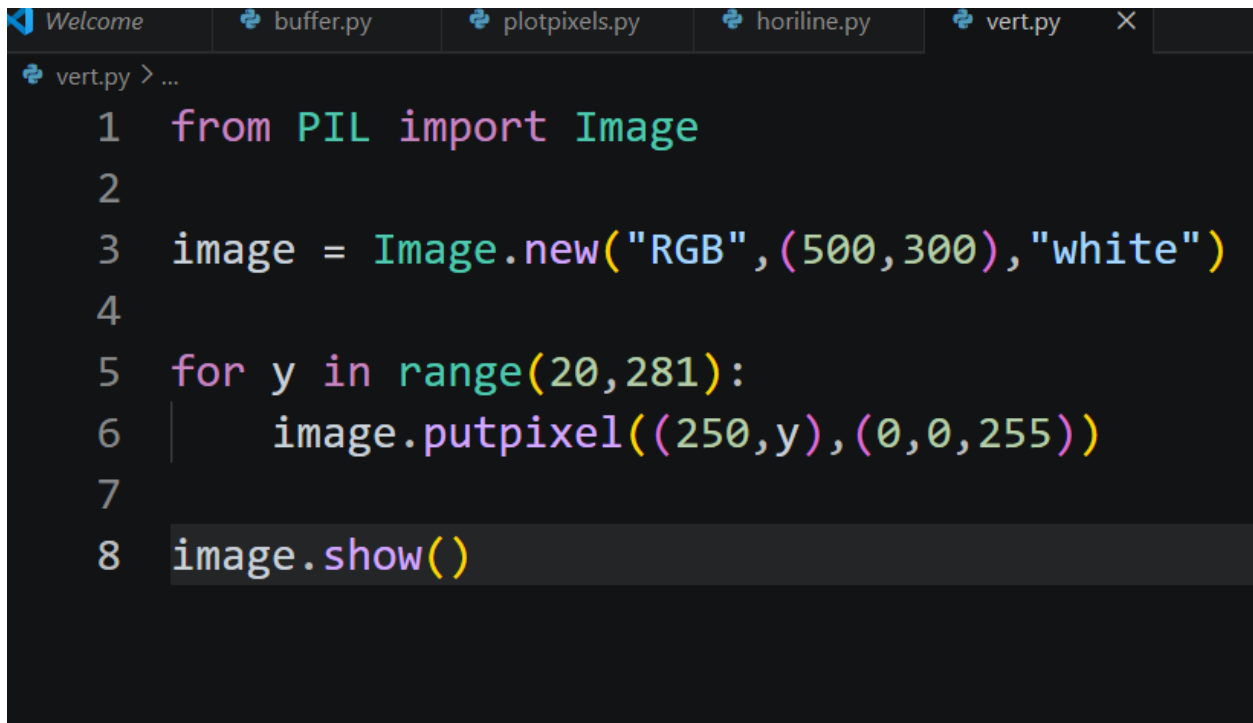
Practical 3: Drawing Horizontal Line Using Pixels

```
horiline.py > ...
1  from PIL import Image
2
3  image = Image.new("RGB", (500,300), "white")
4
5  for x in range(50,451):
6      image.putpixel((x,150),(255,0,0))
7
8  image.show()
```

Explanation

This program creates a **500 × 300** white image and uses a **for loop** to draw a horizontal red line. The loop changes the **x-coordinate** from **50 to 450** while keeping the **y-coordinate fixed at 150**, placing a red pixel at each position using `putpixel()`. As a result, a straight red horizontal line appears across the center of the image, and `image.show()` displays the completed drawing.

Practical 4: Drawing Vertical Line

A screenshot of a code editor with a dark background. The editor has several tabs at the top: 'Welcome', 'buffer.py', 'plotpixels.py', 'horiline.py', and 'vert.py'. The 'vert.py' tab is active. The code in the editor is as follows:

```
vert.py > ...
1  from PIL import Image
2
3  image = Image.new("RGB", (500, 300), "white")
4
5  for y in range(20, 281):
6      image.putpixel((250, y), (0, 0, 255))
7
8  image.show()
```

Explanation

This program creates a **500 × 300** white image and uses a **for loop** to draw a vertical blue line. The loop changes the **y-coordinate** from **20 to 280** while keeping the **x-coordinate fixed at 250**, placing a blue pixel at each position using `putpixel()`. This creates a straight vertical line in the middle of the image, and `image.show()` displays the completed drawing.

Practical 5: Drawing Rectangle Using Pixels

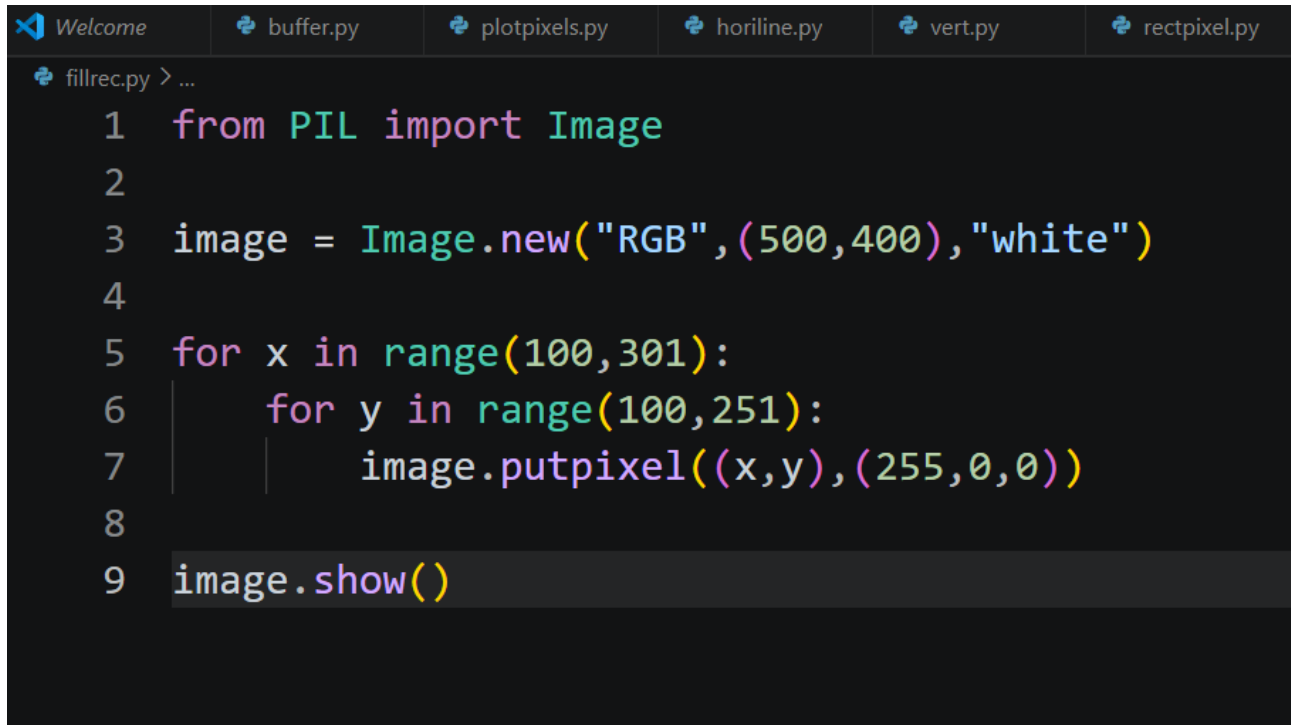
```
rectpixel.py > ...
1  from PIL import Image
2
3  image = Image.new("RGB", (500, 400), "white")
4
5  # Top
6  for x in range(100, 300):
7      image.putpixel((x, 100), (0, 0, 0))
8
9  # Bottom
10 for x in range(100, 300):
11     image.putpixel((x, 250), (0, 0, 0))
12
13 # Left
14 for y in range(100, 251):
15     image.putpixel((100, y), (0, 0, 0))
16
17 # Right
18 for y in range(100, 251):
19     image.putpixel((300, y), (0, 0, 0))
20
21 image.show()
```

Explanation

This program creates a **500 × 400** white image and uses four **for loops** to draw the outline of a rectangle. The first loop draws the **top horizontal line**, the second draws the **bottom horizontal line**, the third draws the **left vertical line**, and the fourth draws the **right vertical line**. Each pixel is colored **black** (0, 0, 0) using the `putpixel()` function.

The horizontal lines are drawn by changing the **x-coordinate** while keeping the **y-coordinate** fixed, whereas the vertical lines are drawn by changing the **y-coordinate** while keeping the **x-coordinate** fixed. Together, these four lines form a complete rectangular outline, and `image.show()` displays the finished rectangle on the screen.

Practical 6: Drawing Filled Rectangle

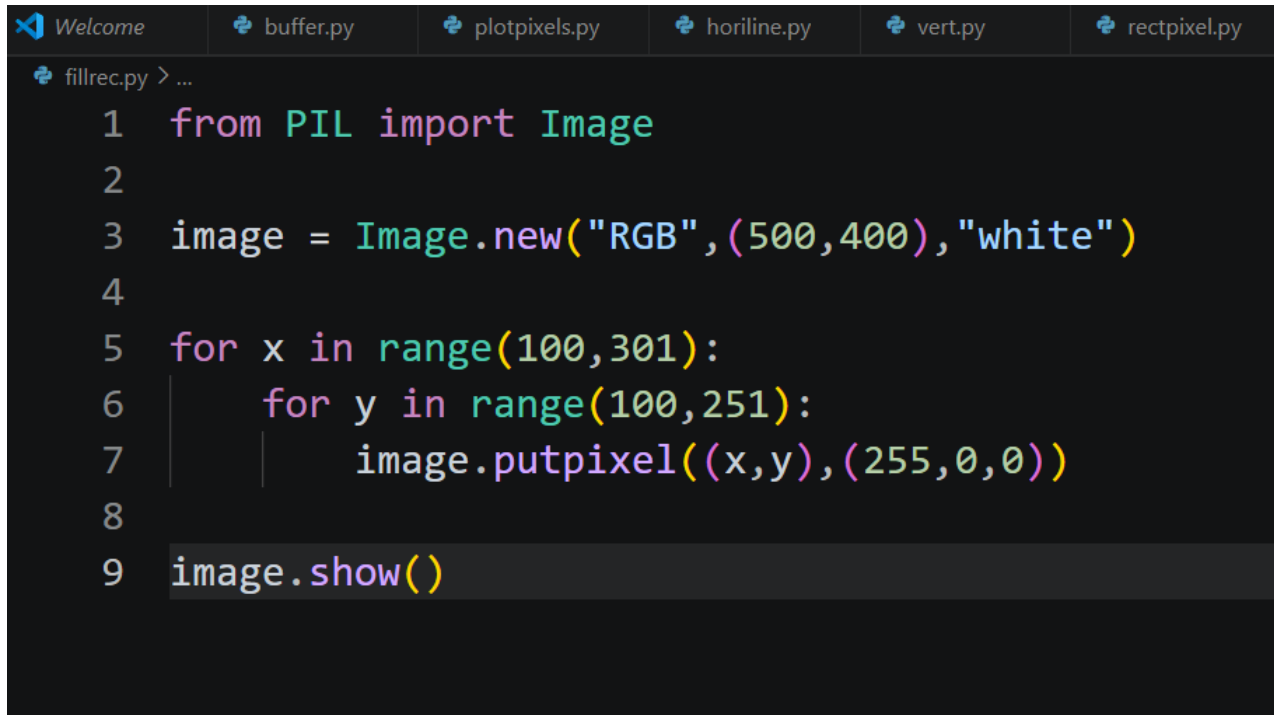
A screenshot of a code editor with a dark background. At the top, there is a tab bar with several tabs: 'Welcome', 'buffer.py', 'plotpixels.py', 'horiline.py', 'vert.py', and 'rectpixel.py'. The 'fillrec.py' tab is active, and the cursor is at the end of the first line. The code is as follows:

```
1 from PIL import Image
2
3 image = Image.new("RGB", (500, 400), "white")
4
5 for x in range(100, 301):
6     for y in range(100, 251):
7         image.putpixel((x, y), (255, 0, 0))
8
9 image.show()
```

Explanation

This program creates a 500×400 white image and uses two nested **for loops** to fill a rectangular area with the color red. The outer loop controls the **x-coordinate** (horizontal movement), while the inner loop controls the **y-coordinate** (vertical movement), placing red pixels at every position between the given ranges. The `putpixel()` function changes each selected pixel to red (255,0,0), creating a filled red rectangle, and `image.show()` displays the final image.

Practical 7: Drawing Diagonal Line

A screenshot of a Python IDE with a dark theme. The top bar shows several open files: 'Welcome', 'buffer.py', 'plotpixels.py', 'horiline.py', 'vert.py', and 'rectpixel.py'. The active file is 'fillrec.py'. The code in the editor consists of nine lines: line 1 imports 'Image' from 'PIL'; line 2 is a blank line; line 3 creates a new 500x400 white image; line 4 is a blank line; lines 5 and 6 are nested 'for' loops for x and y coordinates; line 7 is an indented 'putpixel' call; line 8 is a blank line; and line 9 calls 'show()' to display the image.

```
1 from PIL import Image
2
3 image = Image.new("RGB", (500, 400), "white")
4
5 for x in range(100, 301):
6     for y in range(100, 251):
7         image.putpixel((x, y), (255, 0, 0))
8
9 image.show()
```

Explanation

This program creates a **500 × 400** white image and uses two nested **for loops** to fill a rectangular area with the color red. The outer loop controls the **x-coordinate** (horizontal movement), while the inner loop controls the **y-coordinate** (vertical movement), placing red pixels at every position between the given ranges. The `putpixel()` function changes each selected pixel to red `(255, 0, 0)`, creating a filled red rectangle, and `image.show()` displays the final image.

Practical Assignment:

Computer Graphics Using Python Pillow

Topics Covered: Frame Buffer, Pixels, Pixel Plotting, Lines, Basic Shapes, and Color Models

Programming Language: Python

Library: Pillow (PIL)

PRACTICAL ASSIGNMENT 1

Creating and Understanding a Frame Buffer

Question 1

Write a Python program using the Pillow library to create a **600 × 400 pixel image** that represents a frame buffer.

Your program should:

1. Create a blank image with a white background.
2. Display the image.
3. Save the image as:

`frame_buffer.png`

Requirements:

- Use `Image.new()`
- Define image width and height using variables.
- Add comments explaining each step.

PRACTICAL ASSIGNMENT 2

Understanding Pixels and Coordinates

Question 2

Create a Python program that generates a **400 × 300 image** and plots individual pixels at different coordinates.

Your program should plot:

Pixel Position Colour

(50,50)	Red
(150,100)	Blue
(250,200)	Green
(350,250)	Black

Requirements:

- Use `putpixel()`
- Use RGB colour values.
- Display the final image.

PRACTICAL ASSIGNMENT 3

Creating Visible Pixels Using Pixel Blocks

Question 3

A single pixel is too small to observe. Write a Python program that creates **large visible pixels**.

Your program should:

1. Create a 400 × 300 image.
2. Create a function called:

`draw_pixel()`

3. Allow the function to draw a pixel block of size 20×20 .
4. Draw the following:
 - Red pixel at coordinate (50,50)
 - Blue pixel at coordinate (150,100)
 - Green pixel at coordinate (250,150)

PRACTICAL ASSIGNMENT 4

Drawing Lines Using Pixels

Question 4

Using individual pixels, create a program that draws the following:

A. Horizontal Line

- Starting point: (50,100)
- Ending point: (350,100)
- Colour: Red

B. Vertical Line

- Starting point: (200,50)
- Ending point: (200,250)
- Colour: Blue

C. Diagonal Line

- Starting point: (50,50)
- Ending point: (250,250)
- Colour: Green

Requirement:

Do not use `ImageDraw.line()`.

Students must manually plot the pixels.

PRACTICAL ASSIGNMENT 5

Drawing Rectangles Using Pixels

Question 5

Develop a Python program that creates rectangles using individual pixels.

Your program should draw:

Rectangle 1

- Position:

(50, 50) to (200, 150)

- Colour: Red

Rectangle 2

- Position:

(250, 100) to (400, 250)

- Colour: Blue

Requirements:

- Use loops.
- Use `putpixel()`.
- Draw only the borders of the rectangles.

PRACTICAL ASSIGNMENT 6

Creating Filled Shapes Using Pixels

Question 6

Write a Python program that creates a filled rectangle using pixels.

The rectangle should:

- Start at coordinate:

(100,100)

- End at:

(300,250)

- Fill colour:

RGB (255,165,0)

(Orange)

Requirements:

Use:

- Nested loops
- `putpixel()`

PRACTICAL ASSIGNMENT 7

Exploring RGB Color Models

Question 7

Create a Python program that demonstrates the RGB colour model.

Your program should create three colour sections:

Section 1

Red intensity increases from:

0 → 255

Section 2

Green intensity increases from:

0 → 255

Section 3

Blue intensity increases from:

0 → 255

Display the final colour gradient image.

FINAL PRACTICAL PROJECT

Pixel Art Design Using Python Pillow

Question 8: Creating the AMFEX Logo Using Pixels

Using only:

- Pixels
- Squares
- Rectangles
- Loops
- RGB Colours

Create a pixel-based design that displays the word:

AMFEX

Requirements:

Students must:

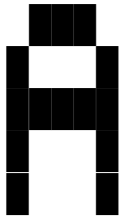
1. Create a canvas of:

800 × 400 pixels

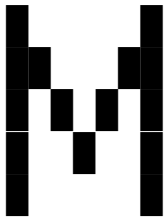
2. Use small square blocks as pixels.
3. Design each letter manually using rectangles.

Example:

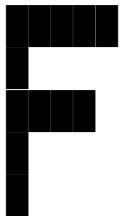
Letter A:



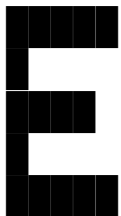
Letter M:



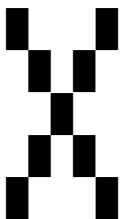
Letter F:



Letter E:



Letter X:



Submission Instructions

- The final output must be colourful.
- The submission deadline is **a week today, 8th August, 2026**.
- Students must take screenshots of their codes and insert the corresponding image output directly below each code screenshot in a Microsoft Word document.
- The Word document must be named using the student's **Index Number**.
- Copied work will attract **zero (0) marks for both students involved**.